

Vault

A self-hosted secure document vault

Report date	19 August 2026
Status	Operational — migrated to Windows / WSL2 and verified
Backend	NestJS · Fastify · Prisma · PostgreSQL 16.14
Surfaces	Client website · Administrator console · Flutter client (deferred)
Test suite	117 tests across 10 suites — all passing
Code	~13,300 lines across backend, web clients and edge

Contents

1. What Vault is	
2. The two security layers	
3. System architecture	
4. Technology stack	
5. Data model	
6. Document storage and cryptography	
7. Security threat model	
8. Infrastructure leakage prevention	
9. API surface	
10. Screenshots	
11. Current deployment	
12. Migration record	
13. Testing and verification	
14. Known limitations	
15. Operations runbook	
16. Master Prompt	

1. What Vault is

Vault is a self-hosted system for storing highly sensitive personal documents — identity papers, legal deeds, property and financial records — and making them readable remotely by a small number of explicitly authorised people, while revealing nothing about the machine that holds them.

It runs on hardware the owner controls. Documents are encrypted at rest with per-document keys. Authorised people reach it through a public address that carries only the client website; the administrator surface never leaves the host machine.

The problem it solves

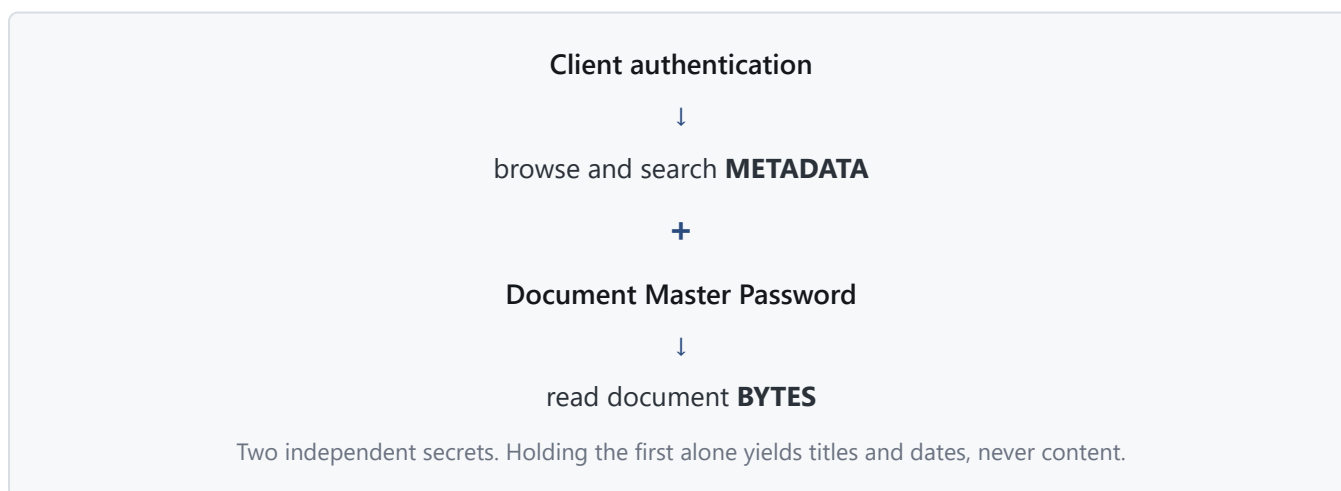
Consumer cloud storage requires trusting a third party with the plaintext of documents that would be catastrophic to lose control of. Self-hosting removes that third party but introduces a new exposure: the address, and therefore the household, becomes discoverable. Vault is built so that being able to reach the service reveals nothing about where it is or what it runs on.

Design premises

- **Being signed in is not permission to read a document.** Authentication buys metadata only; document bytes need a second, separate secret.
- **The client is untrusted.** It holds no secrets, makes no authorisation decisions, and contains no code path that could reveal one.
- **Absence is the guarantee.** Where a capability must not exist, it is not present at all rather than permission-gated — and automated checks fail the build if it ever appears.
- **Fail closed.** Until the Document Master Password is configured, no document content is readable by anyone, including the administrator.

2. The two security layers

Everything in the codebase follows from one rule.



Why the layers are separate

A stolen phone, a hijacked session or a compromised client device defeats layer one only. The attacker sees that a document called "Partnership Deed" exists and nothing more. Reading it additionally requires a password that the client never stores, never caches and cannot change.

The master password is administrator-only

Clients may *submit* it for verification and nothing else. There is no route anywhere in the application that lets a client set, change, reset, recover or read it. This is enforced three ways: no such endpoint exists; a route-coverage test walks the live router and fails the build if anything resembling password management appears outside the administrator surface; and a string check fails the build if a “forgot password” affordance appears in any client-facing file.

Unlock grants are bound and short-lived

A successful unlock issues an opaque random token bound to the client, the device, the session and the current master-password version. It expires on a timer and is held in memory only. Changing the master password increments its version, which invalidates every outstanding grant on the next request.

3. System architecture

Components

COMPONENT	ROLE	PORT	EXPOSURE
backend/	The API. All authorisation, crypto and audit logic.	3000	Loopback only
edge/server.mjs	Single front door: serves the site, proxies the API, hides admin routes.	8080	Public via tunnel
web-client/	The website authorised people sign in to.	via 8080	Public via tunnel
admin-web/	Administrator console.	5173	Host machine only
client-app/	Flutter client for iOS/Android. Written and tested; deployment deferred.	—	—

The front door

A single process on port 8080 serves the built website and proxies `/api/` to the backend, so the site and the API always share one origin. It refuses anything under `/api/v1/admin` with a flat `404` before the request reaches the API — so exposing the website to the internet does not expose the administrator surface with it. The refusal is indistinguishable from any other unmatched path: there is no hint that an administrator surface exists at all.

It listens on both IPv4 and IPv6 loopback. `localhost` resolves to `::1` first on most systems, and a single IPv4 listener produces intermittent connection failures that are painful to diagnose.

Guard chain

Every protected request passes the same layered checks, in order:

GUARD	ASSERTS
route-scope.guard	The route declares a scope at all — an undeclared route is a build failure, not an open door.
client-auth.guard	A valid, unexpired session token.
device-signature.guard	The request is signed by the enrolled device key, covering method, path, timestamp and body hash.
device-status.guard	The device has not been revoked.
document-resource.guard	This client may see this specific document — otherwise <code>404</code> , never <code>403</code> .
document-grant.guard	A live master-password grant exists for document bytes.
rate-limit.guard	Per-device, per-client and global limits with backoff and lockout.

4. Technology stack

LAYER	CHOICE	WHY
API framework	NestJS on Fastify	Guards and decorators make the authorisation chain declarative and testable in isolation.
Database	PostgreSQL 16	Full-text search for document metadata; strict constraints; durable audit chain.
ORM	Prisma	Typed queries and a migration history that travels with the dump.
Password hashing	argon2id	m=64 MiB, t=3, p=4 — calibrated to roughly 500 ms per verification.
Document encryption	AES-256-GCM	Authenticated encryption; a truncated or tampered blob fails to decrypt rather than returning garbage.
Device identity	ECDSA P-256	Chosen so a later move to Secure Enclave / StrongBox is a re-enrolment, not a wire-format change.
Web clients	React + Vite	Small, no server-side rendering, no framework banner in responses.
Mobile client	Flutter	One codebase for iOS and Android with platform-backed key storage.
Public access	Cloudflare Tunnel	Outbound-only. No router port forwarding, no fixed IP, origin address never in DNS.

5. Data model

Eighteen tables. The security-relevant ones:

TABLE	HOLDS
<code>vault_secret</code>	Single-row table (enforced by check constraint) holding the master-password hash, its version, and the grant lifetime.
<code>documents</code>	Metadata plus the wrapped per-document key: <code>enc_dek</code> , <code>enc_dek_nonce</code> , <code>enc_dek_tag</code> , and a server-generated <code>storage_key</code> .
<code>document_grants</code>	Live unlock grants — hashed token, client, device, session, master-password version, expiry.
<code>dmp_attempts</code>	Durable record of master-password attempts, so brute-force protection survives a restart.
<code>devices</code>	Enrolled device public keys and revocation status.
<code>enrollment_codes</code>	Single-use, time-limited device setup codes.
<code>clients</code> / <code>client_category_access</code>	People and the categories each may see. Nothing is visible until granted.
<code>audit_log</code>	Append-only, hash-chained event log with a verification endpoint.
<code>admins</code> / <code>admin_sessions</code>	Administrator accounts with TOTP, and their short-lived sessions.

Categories

Access is granted per category, not per document, so adding a document to a category the person already holds does not require re-granting. The nine configured categories are: Personal, Education, Work, Finance, Government, Property, Vehicle, Legal, Other.

6. Document storage and cryptography

Envelope encryption

Each document gets its own randomly generated 256-bit key (a DEK). The document is encrypted with it using AES-256-GCM. The DEK is then wrapped with a server-held key-encryption key (the KEK, `STORAGE_KEY`) and the wrapped form is stored in the database. The plaintext DEK is never written to disk.

Write path — administrator upload

```
Stream to a private temp dir on the same filesystem
-> enforce size cap while streaming, abort past the cap
-> sniff real MIME from magic bytes, IGNORE the client's extension
-> reject unless allowlisted: pdf png jpeg webp heic docx xlsx txt
-> sanitise filename for DISPLAY only, never as a path component
-> compute sha256
-> random 256-bit DEK -> AES-256-GCM encrypt -> write to storage_key
-> wrap DEK with KEK -> store enc_dek / nonce / tag in the database
-> insert metadata, index for search, write audit entry, delete temp file
```

Path traversal is structurally impossible: the supplied filename is never a path component, and `storage_key` is generated server-side from a UUID. An executable renamed `.pdf` is rejected on its magic bytes — a test asserts exactly this.

The KEK is irreplaceable

`STORAGE_KEY` unwraps the encryption key of every stored document. `SERVER_PEPPER` is mixed into every password hash. Regenerating either makes every existing document permanently unreadable and every password stop working. There is no recovery path, by design. Three things must be backed up together and are useless apart: the database, the encrypted blobs, and the KEK — the last kept somewhere other than the host machine.

Not zero-knowledge — stated plainly

Document keys are wrapped by a server-held KEK, not derived from the master password. An attacker holding both the server and the KEK can read documents. The master password protects against a compromised client, a stolen session and a lost device — not a compromised server. The alternative was considered and rejected: deriving keys from the master password would make changing it a full re-encryption of every document, and would make losing it equivalent to losing everything.

7. Security threat model

#	ATTACK	CONTROL
1	Inspect API requests	TLS; no secrets in transit beyond credentials; the master password is never returned.
2	Modify client requests	All authorisation server-side; device signature covers method, path, timestamp and body hash.
3	Reverse-engineer the client	Client holds no secrets, no master password, no comparison logic, no admin routes, no storage paths.
4	Modify local client storage	An access token alone is insufficient — signing requires the device private key.
5	Call the API manually	Identical guard chain regardless of caller; no endpoint trusts a client-set flag.
6	Replay requests	Nonce cache plus a ± 60 s timestamp window; refresh rotation with reuse detection.
7	Brute-force the master password	Argon2id at ~ 500 ms, layered rate limits, lockout, durable attempt table. The attacker must already hold a valid device session.
8	Guess document IDs (IDOR)	Random UUIDs, explicit per-request resource guard, and <code>404</code> rather than <code>403</code> .
9	Path traversal	Client filenames are never path components; storage keys are server-generated.
10	Malicious upload	Admin-only ingestion, magic-byte detection, allowlist, size cap, no execution, storage outside any web root.
11	Inspect HTTP errors	Single sanitised envelope, fixed code enum, uniform 404s.
12	Inspect response headers	No framework banners, no version headers.
13	Discover infrastructure	See section 8.
14	Steal an unlock grant	Header-only so CSRF does not apply; memory-only on the client; bound to device and session; absolute expiry.
15	Compromised admin account	TOTP, step-up re-authentication for every destructive action, lockout, full audit trail, separate network exposure.
16	Timing side channel	Argon2 verification always runs, including a dummy verify when no hash exists; constant error shape.

What this design does not defend against

A fully compromised server with KEK access can read every document. A rooted or jailbroken device with the app unlocked can exfiltrate whatever the user is currently viewing. Network observers can see that a TLS connection exists to a hostname and IP. This design minimises information leakage; it does not promise invisibility.

8. Infrastructure leakage prevention

The client must never learn anything about the machine behind the service. No server status, IP, hostname, location, operating system, filesystem path, database detail or hosting information may appear in a client response.

How it is enforced

- A single global exception filter maps every internal cause to a sanitised public code. A refused database connection, a missing storage file and a document that never existed all produce the same `404` .
- Unauthorised and nonexistent are indistinguishable — always `404` , never `403` .
- No `/health` , `/metrics` , `/version` or `/status` on the public listener.
- No directory listing; no static file middleware registered at all on the API.
- CORS admits exactly one origin — the administrator console. The client API allows none.
- The client has no status, connectivity or diagnostic UI of any kind. Every transport failure produces the same generic screen.
- A build-failing check greps for banned vocabulary — *server*, *offline*, *connection refused*, *host*, *unreachable*, *timeout* — in every client-facing string, and for any client-side password-reset path.

Error mapping

INTERNAL REALITY	HTTP	PUBLIC CODE	LOGGED INTERNALLY
Database connection refused	404	<code>NOT_FOUND</code>	full error + stack + request id
Storage path missing / permission denied	404	<code>NOT_FOUND</code>	full error
Document does not exist	404	<code>NOT_FOUND</code>	debug
Document exists, client not permitted	404	<code>NOT_FOUND</code>	warn + audit
Valid session, no unlock grant	401	<code>GRANT_REQUIRED</code>	info

9. API surface

All routes sit under `/api/v1`. Every route carries an explicit scope declaration; a route without one fails the build.

Client — authentication and devices

POST	<code>/auth/enroll</code>	exchange a setup code for a device key
POST	<code>/auth/login</code>	account + password -> session
POST	<code>/auth/refresh</code>	rotate the session token
POST	<code>/auth/logout</code>	
GET	<code>/auth/me</code>	

Client — documents (metadata; layer one only)

GET	<code>/documents</code>	list and search (metadata only)
GET	<code>/documents/:id</code>	one document's metadata
POST	<code>/documents/:id/favorite</code>	
DELETE	<code>/documents/:id/favorite</code>	

Client — document bytes (requires a live grant)

GET	<code>/documents/:id/content</code>	the decrypted document
GET	<code>/documents/:id/preview</code>	inline preview stream
GET	<code>/vault/status</code>	is the vault configured and unlocked
POST	<code>/vault/unlock</code>	submit the master password -> grant
POST	<code>/vault/lock</code>	discard the grant

Administrator — never reachable through the tunnel

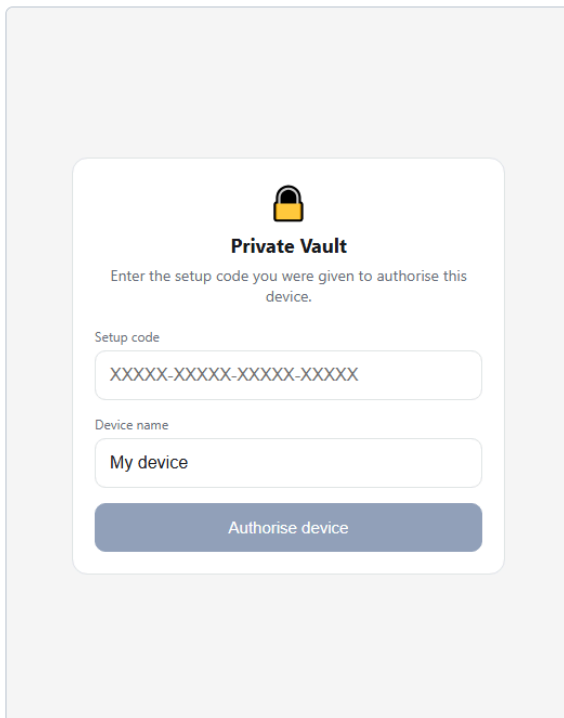
POST	<code>/admin/auth/login</code>	password + TOTP
POST	<code>/admin/auth/stepup</code>	re-authentication for destructive actions
POST	<code>/admin/auth/logout</code>	
GET	<code>/admin/dashboard</code>	
GET	<code>/admin/categories</code>	
GET	<code>/admin/clients</code>	POST to create
PUT	<code>/admin/clients/:id/access</code>	grant categories
POST	<code>/admin/clients/:id/enrollment-code</code>	
GET	<code>/admin/clients/:id/devices</code>	
POST	<code>/admin/clients/:id/revoke</code>	
POST	<code>/admin/devices/:id/revoke</code>	
GET	<code>/admin/documents</code>	POST /upload, PATCH /:id, DELETE /:id
GET	<code>/admin/security/master-password</code>	status only, never the value
POST	<code>/admin/security/master-password set</code>	(step-up required)
PUT	<code>/admin/security/master-password change</code>	(step-up required)
PUT	<code>/admin/security/grant-ttl</code>	
POST	<code>/admin/security/invalidate-all-grants</code>	
GET	<code>/admin/security/audit-log</code>	and /audit-log/verify

Verified behaviour

Every path above beginning `/admin` returns `404` when requested through the public front door, across GET, POST, PUT and DELETE — confirmed against the live public address, not simulated locally.

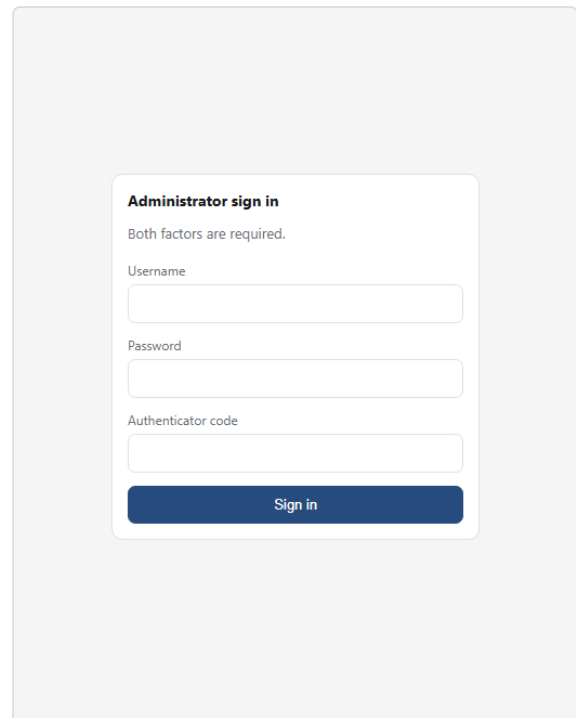
10. Screenshots

Captured from the running system on 19 August 2026.



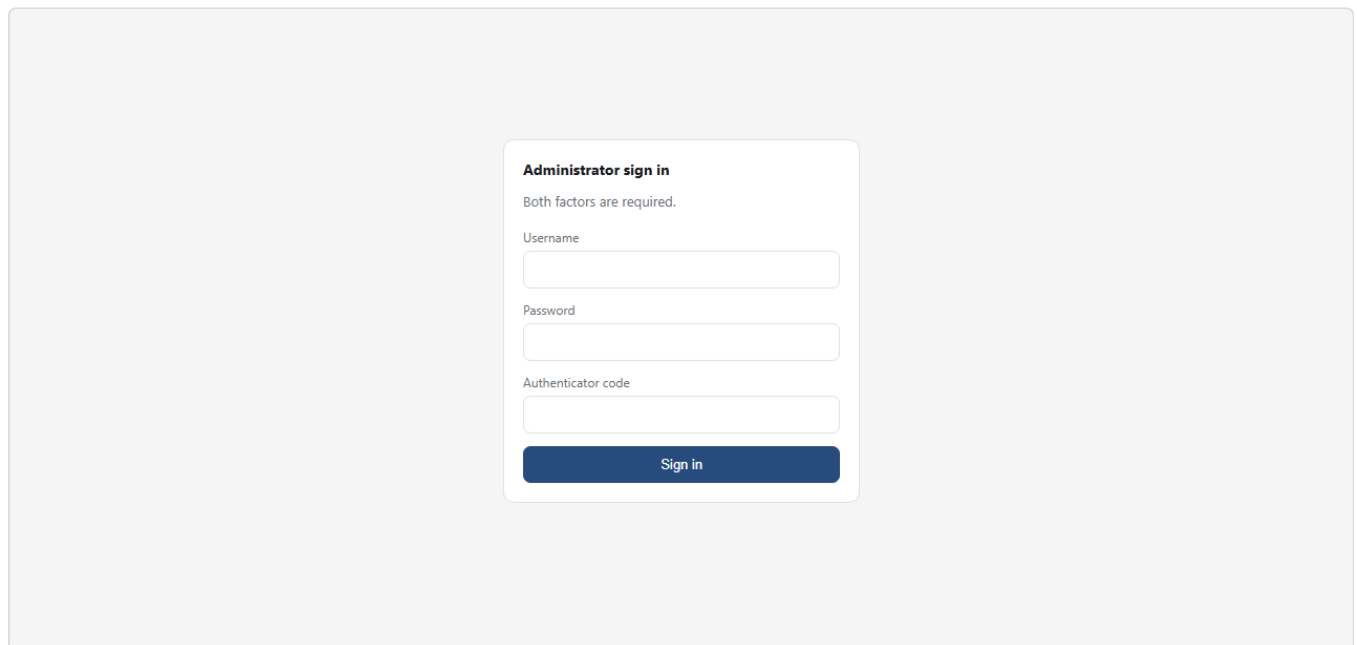
The screenshot shows a 'Private Vault' enrolment screen. At the top is a yellow padlock icon. Below it, the title 'Private Vault' is centered. The instruction 'Enter the setup code you were given to authorise this device.' is displayed. There are three input fields: 'Setup code' with a placeholder 'XXXXX-XXXXX-XXXXX-XXXXX', 'Device name' with the text 'My device', and a large blue 'Authorise device' button at the bottom.

Client website — device enrolment. A browser with no device key sees only this. The setup code is single-use and time-limited. Note the absence of any “forgot password” affordance.



The screenshot shows an 'Administrator sign in' screen. The title is 'Administrator sign in'. Below it, the text 'Both factors are required.' is displayed. There are three input fields: 'Username', 'Password', and 'Authenticator code'. A large blue 'Sign in' button is at the bottom.

Administrator console at 520 px. The console is desktop-first but reflows: the sidebar becomes a horizontal bar and wide tables scroll inside their own card.



This screenshot shows the 'Administrator sign in' screen at a width of 520px. The layout is more compact than the desktop version. The title 'Administrator sign in' is at the top, followed by 'Both factors are required.'. The input fields for 'Username', 'Password', and 'Authenticator code' are stacked vertically. A blue 'Sign in' button is at the bottom.

Administrator console — sign in. Both factors are required: password and authenticator code. The session token is held in a module variable only — not in `localStorage`, not in a cookie — so a page refresh signs the administrator out. That is a real cost, accepted deliberately for the one console that can change the Document Master Password.

Pretty-print ☐

```
{"error":{"code":"NOT_FOUND","requestId":"edge"}}
```

The administrator API through the public door. A request to `/api/v1/admin/security/master-password` returns a flat `404` carrying only a fixed error code. `requestId: "edge"` shows the front door refused it before the API was ever consulted. Nothing distinguishes this from any other unmatched path.

11. Current deployment

Vault originally ran on an ARM Mac. It now runs on a Windows desktop inside WSL2, which lets the original bash tooling run unchanged.

Host

ITEM	VALUE
Machine	Lenovo desktop, Intel Core i5-2300, 8 GB RAM
Host OS	Windows 10 Pro, build 19045
Guest	Ubuntu 24.04 LTS under WSL2, kernel 5.10.16
Project location	<code>~/vault</code> in the Linux filesystem — <i>not</i> on the Windows drive
Linux disk	On the SSD; builds, Postgres and the test suite are all I/O-heavy
Node.js	v24.19.0 (source machine ran v24.18.0)
PostgreSQL	16.14 (dump was taken from 16.14)
cloudflared	2026.8.2

Live state

METRIC	AT MIGRATION	NOW
Documents	2	2
Clients	1	1
Enrolled devices	2	5
Audit entries	19	41
Master-password version	1	2 (changed since)
Unlock window	60 s	900 s

Codebase

AREA	FILES	LINES
<code>backend/src</code>	57	5,569
<code>backend/test</code>	13	2,520
<code>client-app/lib</code> (Flutter)	19	2,464
<code>admin-web/src</code>	9	1,529
<code>web-client/src</code>	4	1,066
<code>edge</code>	1	138

Port conflict on this machine

Two unrelated projects run on Windows and occupy Vault's ports: a Next.js dev server for `E:\SugarRush\icecream 2` on 3000, and a Vite dev server for `E:\PrimeGold` on 5173. They bind `::`, and `localhost` resolves to `::1` first, so `localhost:5173` opens PrimeGold, not the Vault console. Use `127.0.0.1:5173` instead, or stop those servers. Port 8080 is unaffected.

12. Migration record

What was done to bring the vault up on this machine, and what was verified at each step.

STEP	OUTCOME
Enable virtualisation	The CPU supported VT-x but it was disabled in firmware, and the BIOS key was not catchable at boot. Set through Windows instead via the Lenovo WMI BIOS interface, then verified by reading the setting back.
Install WSL2 + Ubuntu	The Microsoft Store distribution download failed on a DNS error; Ubuntu 24.04's official rootfs was downloaded directly and imported with <code>ws1 --import</code> .
Copy the project	Verified byte-identical: matching MD5s for <code>.env</code> , both encrypted blobs and the SQL dump; a full recursive <code>diff</code> reported no differences.
Restore the database	Row counts matched <code>expected-counts.txt</code> exactly. Master password confirmed present. Both storage keys matched files on disk.
Configure database access	Rather than loosening <code>pg_hba.conf</code> to trust loopback as the Mac had, the role was given a generated password under Ubuntu's default <code>scram-sha-256</code> — stronger than the source machine.
Edit <code>.env</code>	Only the two database URL lines changed. The MD5 fingerprint of <code>JWT_SECRET</code> , <code>SERVER_PEPPER</code> , <code>STORAGE_KEY</code> and <code>IP_HASH_SALT</code> was identical before and after.
Install and build	<code>prisma generate</code> only — never <code>migrate dev</code> . The <code>argon2</code> native module resolved to the correct <code>linux-x64</code> prebuilt and was verified by hashing and round-tripping a value.
Verify	117 tests across 10 suites passed; the leakage check passed; admin routes returned 404 through the live public address.

Changes made after migration

- **Responsive fix, administrator console.** The Manage panel was a fixed, centred overlay with no scroll container, so once the device list made it taller than the window the controls below the fold — including “Issue device setup code” — could not be reached. The overlay now scrolls; two breakpoints were added; wide tables scroll inside their card.
- **Same defect closed on the client website.** Its master-password sheet had identical structure and would clip in landscape.
- **Documentation.** `RUNNING.md` was rewritten for this machine; the macOS original is preserved as `RUNNING-macos.md`.

13. Testing and verification

The suite is deliberately adversarial — it asserts what must *not* be possible, not merely that the happy path works.

SUITE	ASSERTS
document-access	A valid session cannot read document bytes without a grant; one client's grant is useless on another's device.
dmp	Changing the master password locks every client out on the next request; failures are constant in shape and timing.
envelope	A truncated or tampered encrypted blob never decrypts.
upload	An executable renamed <code>.pdf</code> is rejected on its magic bytes.
auth	A revoked device stops working immediately; replayed requests are refused.
admin-separation	The administrator surface is unreachable from a client session.
route-coverage	Walks the live router; fails if any route lacks a scope declaration, or if anything resembling password management appears outside the admin surface.
leakage	Sweeps every endpoint; fails if any response contains a driver name, framework name, filesystem path or socket error.
client-interop	The Flutter client's real key store signs a request and the backend's real verifier checks it — catching cross-language signature drift.
favourites	A document that exists but is not permitted returns the same 404 as one that never existed.

```
Test Suites: 10 passed, 10 total
Tests:      117 passed, 117 total
Time:       ~93 s
```

```
check-forbidden-strings.sh
-> checking the public error-code vocabulary      ok
-> checking client-facing strings                 ok
-> checking the client has no master-password mgmt ok
PASS - no infrastructure vocabulary on any client-facing surface.
```

The interop suite caught a genuine defect during development: the client's first crypto library threw `UnimplementedError` from pure-Dart ECDSA key generation, meaning no device could ever have enrolled. Static analysis was clean.

14. Known limitations

An honest list of what is deliberately not done, or not yet working.

LIMITATION	DETAIL
PDF preview fails on Android	Preview renders images with <code></code> and everything else in an <code><iframe></code> . Android Chrome has no inline PDF viewer, so a PDF shows a download placeholder instead of the document. This is the deferred server-side rasterisation noted in the architecture; rendering client-side with PDF.js would fix it without adding file-parsing surface to the server.

LIMITATION	DETAIL
Office formats do not preview	<code>docx</code> and <code>xlsx</code> are accepted for storage but cannot render in a browser without conversion.
Not zero-knowledge	An attacker with both the server and the KEK can read documents.
Device key is software-held	Protected at rest by the platform key store, but extractable on a rooted device. The protocol already uses P-256 so moving to Secure Enclave / StrongBox is a re-enrolment, not a format change.
Rate limits are per-process	Correct for a single node; a multi-instance deployment would need shared state. Master-password brute-force protection does not depend on this — it is backed by a durable table.
Client uploads not implemented	Admin-only ingestion removes the largest attack surface at no cost to the current use.
Not run on real phone hardware	The Flutter client is rendered headlessly and in a browser; Keychain, StrongBox and <code>FLAG_SECURE</code> behaviour is unexercised.
Public address changes	The free tunnel issues a new hostname on every restart. A named tunnel on an owned domain fixes this.
Cloudflare's own error page	When the vault is stopped, visitors see Cloudflare's Error 1033, which names the tunnel technology. It is generated at Cloudflare's edge and cannot be customised on a free quick tunnel; an owned domain plus a Worker would allow a blank error page.
Host must stay awake	The machine must remain powered on with the start command running. Sleep and hibernate are disabled on this host.

15. Operations runbook

Start

```
wsl -d Ubuntu -- bash -c "sudo service postgresql start && cd ~/vault && ./scripts/vault-up.sh"
wsl -d Ubuntu -- bash -c "sudo service postgresql start && cd ~/vault && ./scripts/vault-up.sh --public"
```

Leave the window open — the open session is what keeps the Linux environment alive. `--public` prints a `trycloudflare.com` address usable from anywhere.

Stop

```
Ctrl-C in that window, or:
wsl -d Ubuntu -- bash -c "pkill -f '[d]ist/main.js'; pkill -f '[e]dge/server.mjs'; pkill -f '[v]ite --port'"
```

The square brackets stop the pattern from matching — and killing — the shell running it.

Verify

```
wsl -d Ubuntu -- bash -c "cd ~/vault/backend && npm test" # 117 tests
wsl -d Ubuntu -- bash -c "cd ~/vault/backend && npm run lint:leakage"
```

Back up — all three, or none are any use

```
wsl -d Ubuntu -- bash -c "sudo service postgresql start && cd ~/vault && ./scripts/vault-export.sh"
```

Writes a fresh database dump into `migration/`, replacing what is there. Copy `migration/`, `backend/var/storage/` and `backend/.env` off the machine together.

Never do these

- Never regenerate `STORAGE_KEY`, `SERVER_PEPPER`, `JWT_SECRET` or `IP_HASH_SALT`.
- Never run `prisma migrate reset` or `prisma migrate dev`.
- Never add a client-facing master-password reset, recovery or change flow.
- Never expose the administrator console through the tunnel.
- Never run the vault on two machines at once — they diverge immediately into two different vaults with incompatible audit chains and device records.

16. Master Prompt

Copy everything in the box below into a fresh session to rebuild this project from nothing. It is written to be self-contained: it states the security model, the non-negotiable rules, the stack, the surfaces and the tests that prove the properties hold.

This rebuilds the software, not this vault

A rebuilt system starts empty, with newly generated secrets. It cannot read the documents stored by this one — those are bound to the existing `STORAGE_KEY`. To move *this* vault, copy the database, the encrypted blobs and `.env` instead.

Build me a self-hosted secure document vault called Vault. I am not a developer; explain what you are doing in plain language and do not leave me at a half-finished step.

PURPOSE

A system I run on my own machine that stores highly sensitive personal documents (identity papers, deeds, financial and legal records) and lets a small number of people I authorise browse and read them remotely from anywhere - while revealing nothing about the machine hosting it.

THE ONE RULE EVERYTHING FOLLOWS FROM

Being logged in is NOT permission to read a document.

Client authentication -> browse and search METADATA only
+ Document Master Password -> read document BYTES

Two independent secrets. Holding the first alone yields titles, categories and dates, never content.

NON-NEGOTIABLE RULES

1. The Document Master Password is administrator-only. Clients may SUBMIT it for verification and nothing else. There must be no route, screen, button or API anywhere that lets a client set, change, reset, recover or view it - not even a disabled one. Its absence is the guarantee. Never add a "forgot password" flow to any client surface.
2. No client response may ever contain server status, IP, hostname, location, OS, filesystem path, database detail, driver name, framework name or hosting information. Never use words like "server offline", "connection failed" or "backend unavailable" in client-facing text. The pattern for anything unreachable, forbidden or nonexistent is a plain 404 - unauthorised and nonexistent must be indistinguishable.
3. Fail closed. Until the master password is configured, no document content is readable by anyone, including the administrator.
4. The administrator console is never exposed publicly. Only the client website goes through the tunnel.
5. Secrets are irreplaceable. `STORAGE_KEY` unwraps every document key and `SERVER_PEPPER` is in every password hash. Regenerating either destroys all documents and passwords permanently. Say so loudly in the docs, and never generate a fresh `.env` to "fix" a missing one.

STACK

Backend: NestJS on Fastify, Prisma, PostgreSQL 16, TypeScript.

Crypto: argon2id for passwords (m=64MiB, t=3, p=4, ~500ms); AES-256-GCM for documents; ECDSA P-256 for device identity.

Clients: React + Vite for the client website and the admin console; Flutter for the phone app.

Edge: one small Node process that serves the built website, proxies /api to the backend so both share one origin, and returns 404 for anything under /api/v1/admin. It must listen on BOTH IPv4 and IPv6 loopback, because localhost resolves to ::1 first and a single IPv4 listener causes intermittent connection failures.

Public: Cloudflare Tunnel - outbound only, no port forwarding, no fixed IP, the origin address never appears in DNS.

DOCUMENT STORAGE (envelope encryption)

Each document gets a random 256-bit key (DEK). Encrypt the document with it using AES-256-GCM. Wrap the DEK with a server-held KEK and store the wrapped form in the database. Never write a plaintext DEK to disk.

Upload path (administrator only):

- stream to a private temp dir on the same filesystem
- > enforce a size cap while streaming, abort past it
- > sniff the real MIME from magic bytes, IGNORE the client's extension
- > reject unless allowlisted: pdf png jpeg webp heic docx xlsx txt
- > sanitise the filename for DISPLAY only, never as a path component
- > compute sha256
- > random DEK -> encrypt -> write to a server-generated UUID storage key
- > wrap DEK with KEK, store enc_dek/nonce/tag
- > insert metadata, index for search, write an audit entry, delete the temp file

Path traversal must be structurally impossible: client filenames are never path components.

GUARD CHAIN - every protected request, in this order

route-scope	the route declares a scope at all; an undeclared route is a BUILD FAILURE, so "someone forgot to protect this endpoint" fails closed
client-auth	valid unexpired session
device-signature	signed by the enrolled device key, covering method, path, timestamp and body hash; nonce cache plus a +/-60s window
device-status	the device is not revoked
document-resource	this client may see this specific document, else 404
document-grant	a live master-password grant exists, for bytes only
rate-limit	per device, per client, and global, with backoff and lockout

UNLOCK FLOW

Client submits the master password -> rate limit -> argon2 verify server-side (always run the verify, including a dummy verify when no hash exists, so timing is constant) -> on success issue an opaque random token bound to client, device, session and the CURRENT master-password version, with an absolute expiry. Store only its hash. Send it in a header, never a cookie, and hold it in memory only on the client. Changing the master password increments the version, which invalidates every outstanding grant on the next request. Failures return an identical body and timing every time.

DATA MODEL

vault_secret (single row, enforced by a check constraint: master-password hash, version, grant TTL), documents (metadata + wrapped DEK + server-generated storage key), document_grants, dmp_attempts (durable, so brute-force protection survives a restart), clients, client_category_access, devices, enrollment_codes (single-use, time-limited), categories, favorites, sessions, admins, admin_sessions, audit_log (append-only, hash-chained, with a verification endpoint).

Access is granted per CATEGORY, not per document. Nothing is visible to a client until a category is granted.

ADMINISTRATOR CONSOLE

Password + TOTP. Hold the session token in a module variable only - not

localStorage, not a cookie - so XSS cannot lift it, CSRF does not apply, and a refresh signs out. Every destructive action (change the master password, delete a document, revoke a client or device, invalidate all grants) requires step-up re-authentication with both factors, however recently the admin signed in. Create the first administrator with a local CLI, never a web "first-run setup" page - that is a race whoever finds the host first can win.

ERROR HANDLING

One global exception filter mapping every internal cause to a sanitised public code:

database connection refused	-> 404 NOT_FOUND	(log fully internally)
storage missing / permission denied	-> 404 NOT_FOUND	
document does not exist	-> 404 NOT_FOUND	
document exists, not permitted	-> 404 NOT_FOUND	(warn + audit)
valid session, no grant	-> 401 GRANT_REQUIRED	

No /health, /metrics, /version or /status on the public listener. No directory listing. No static file middleware on the API. CORS admits exactly one origin - the admin console - and none for the client API.

TESTS - make them adversarial; they are the evidence, not decoration

Assert that a valid session cannot read a document without a grant; that changing the master password locks every client out on the next request; that one client's grant is useless on another's device; that a revoked device stops working immediately; that an executable renamed .pdf is rejected on its magic bytes; that a truncated encrypted blob never decrypts; and that a dropped database or a missing file produces the same 404 as a document that never existed.

Two suites specifically:

route-coverage	- walks the live router and fails the build if any route lacks a scope declaration, or if anything resembling password management appears outside the admin surface.
leakage	- sweeps every endpoint and asserts no response ever contains a driver name, framework name, filesystem path or socket error.

Also a shell check that greps client-facing strings and fails the build on infrastructure vocabulary (server, offline, connection refused, host, unreachable, timeout) or on any client-side password-reset path.

Add a cross-language signing test: the phone client's real key store signs a request and the backend's real verifier checks it. Different languages and crypto libraries default to different ECDSA signature encodings, and the failure mode is a generic 401 on every request with nothing in either log explaining it.

UI

Client website: mobile-first, fluid widths, safe-area insets. Document list is metadata only. Opening a document shows a locked state until the master password is submitted, then Preview and Download appear until the grant expires. Render PDFs with PDF.js in the browser rather than an iframe - Android Chrome has no inline PDF viewer and an iframe shows a download placeholder instead of the document. Any fixed overlay or bottom sheet MUST have its own scroll container, or controls below the fold become unreachable on short screens.

Admin console: sidebar plus main area, reflowing to a horizontal bar on narrow screens; wide tables scroll inside their own card.

DELIVERABLES

- A start script that launches Postgres if needed, builds, starts the API, the edge front door and the admin console, prints the addresses, and on Ctrl-C stops everything and sweeps the ports BY PORT rather than by recorded PID - npm execs its child, so killing npm orphans the real listener, which keeps the port bound while serving nothing.
- An export script that dumps the database into the project so the vault can move

- machines, with a README stating that the dump, the encrypted blobs and the KEK must travel together and that any two without the third are useless.
- A plain-language RUNNING.md for a non-developer owner, and an ARCHITECTURE.md that records the threat model and, honestly, what the design does NOT defend against.

BE HONEST IN THE DOCS

State plainly that this is not zero-knowledge: document keys are wrapped by a server-held KEK, not derived from the master password, so an attacker with the server AND the KEK can read documents. The master password protects against a compromised client, a stolen session and a lost device - not a compromised server. State that network-level metadata cannot be fully hidden: an observer sees the destination IP, the TLS SNI hostname and traffic timing. Minimise leakage; do not promise invisibility.